DevOps CS316 - Assignment #5: CI/CD Pipeline with Git, GitHub Actions and Docker

Member 1: Muhammad Hashir — Reg# 2023429
Member 2: Zain ul Abdeen — Reg# 2023773
Date: May 2026

Course Code: CS316
Instructor: Muhammad Sajid Ali
Total Marks: 100

A. Project Introduction

For this assignment, we successfully containerized a MERN stack application (Task Manager) and implemented an automated Continuous Integration and Continuous Deployment (CI/CD) pipeline. Using GitHub Actions workflows, the pipeline automates the builds, pushes tagged Docker images to the AWS Elastic Container Registry (ECR), and seamlessly deploys them to distinct AWS EC2 instances representing our Testing and Staging environments.

B. Challenges Faced

During the implementation of the CI/CD pipeline and AWS cloud deployment, we encountered and resolved the following key challenges:

- **Windows SSH Key Permissions:** The `.pem` key threw a "Bad permissions" error on Windows when attempting to connect to the EC2 instances. We resolved this using PowerShell `icacls.exe` commands to strip inherited permissions and grant explicit read-only access to the current Windows user, fulfilling the strict SSH key security requirements.
- **AWS Authentication in GitHub Actions:** The CI/CD pipeline needed secure access to push images to ECR. We solved this by creating a dedicated IAM User (`github-actions-deployer`) equipped with the `AmazonEC2ContainerRegistryPowerUser` policy, and securely storing the generated access keys in GitHub Secrets for the workflow to use.

- **Docker Compose Version Syntax:** Automated deployment initially failed on the Ubuntu 24.04 EC2 instances because they defaulted to the modern Docker Compose V2 plugin. We successfully resolved this by updating the GitHub Actions workflow deployment script to use the new `docker compose` command (with a space) instead of the deprecated `docker-compose` format (with a hyphen).
- **React Environment Variables in Docker:** The frontend application required configuration for the public backend API depending on whether it was deployed to Testing or Staging. Since React embeds environment variables at build time, we dynamically resolved this by passing `REACT_APP_API_URL` as a build argument (`--build-arg`) inside our GitHub Actions workflow during the Docker build process.

C. Complete Flow Diagram

The following architecture mapping and workflow table outline the comprehensive CI/CD workflow extending from the developer's initial push to the final staging deployment:

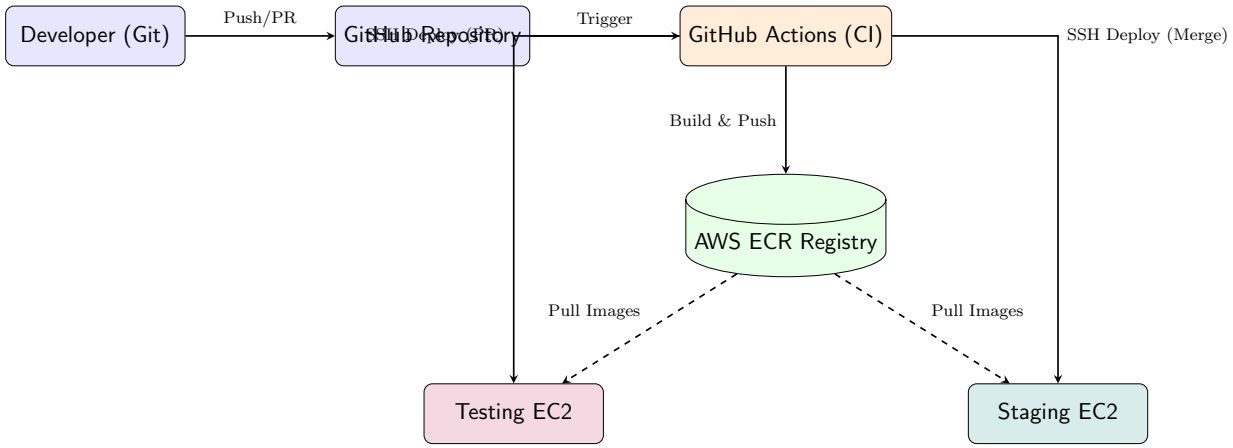

Figure 1: CI/CD Architecture and Automation Flow.

Phase	Workflow Actions & Automation Steps
Developer Workflow	1. Developer creates a feature branch. 2. Code changes committed and pushed to GitHub. 3. A Pull Request (PR) to `main` is opened.
Testing Workflow	1. Triggered automatically on PR creation/update. 2. Source code tested and linted via GitHub Actions. 3. Docker images (Frontend/Backend) built and tagged. 4. Images pushed to AWS ECR. 5. SSH action accesses the Testing EC2 server. 6. Server pulls latest images from ECR and runs `docker compose up -d`. 7. Automated success email sent to QA with testing IP link.
Staging Workflow	1. Triggered automatically on PR merge into `main`. 2. Codebase verified against `main`. 3. Fresh Docker images built and pushed to AWS ECR. 4. SSH action accesses the Staging EC2 server. 5. Server pulls latest images and redeploys containers. 6. Automated success email sent to Team with Staging IP link.

D. Deployment Diagram Description

The application deployment architecture utilizes separate Docker containers running concurrently on an AWS EC2 instance. The React frontend container serves the progressive user interface, while the Node.js backend container handles API integration and business logic. Both application containers securely communicate with a containerized MongoDB database instance over an isolated Docker network, ensuring encrypted data handling, portability, and efficient resource allocation.

E. Screenshots

Task 1: Containerization and Local Setup

Initially, the Task Manager component applications were containerized and verified to run successfully on a local developer machine using `docker-compose`.

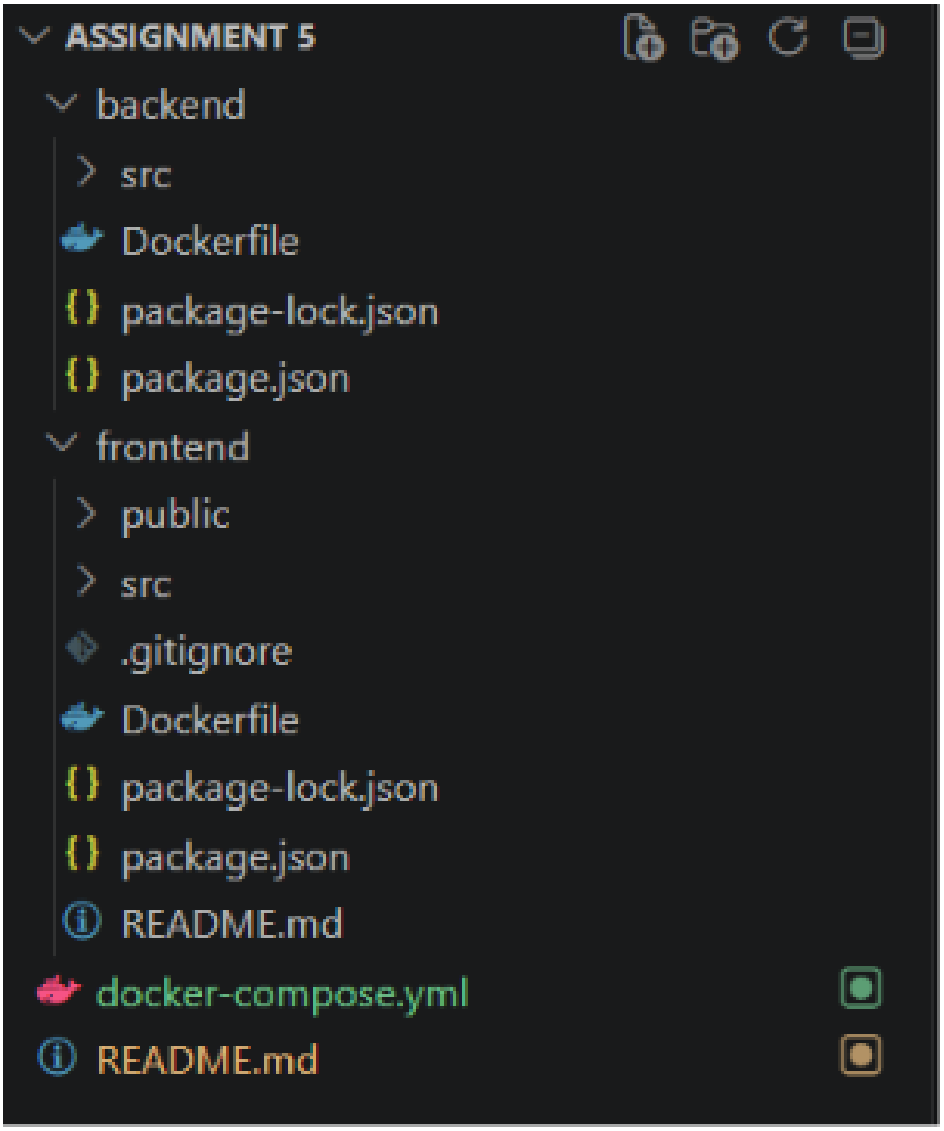

Figure 2: Local Docker containers running via docker-compose.

Task 2: AWS Infrastructure

To support our automated deployment, distinct EC2 environments were provisioned and linked to our Amazon ECR repositories utilizing secure IAM roles.

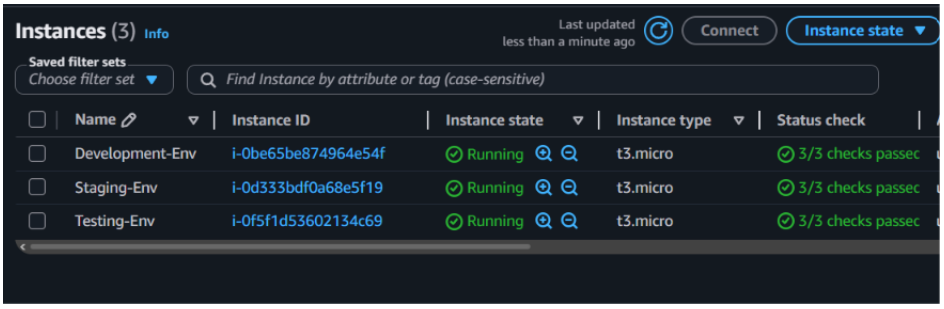

Figure 3: AWS EC2 dashboard showing Dev, Testing, and Staging instances running.

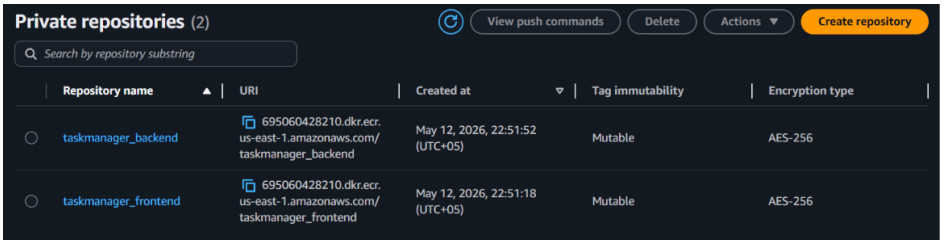

Figure 4: AWS ECR showing the taskmanager_frontend and taskmanager_backend repositories.

Figure 5: AWS IAM role showing the AmazonEC2ContainerRegistryReadOnly policy attached to the EC2 instances.

Task 3: GitHub Actions CI/CD Workflows

The complete pipeline requires extensive identity integration. After securing our GitHub repository with AWS credentials, we validated both the Testing workflow triggered by a PR

and the Staging Workflow upon merging.

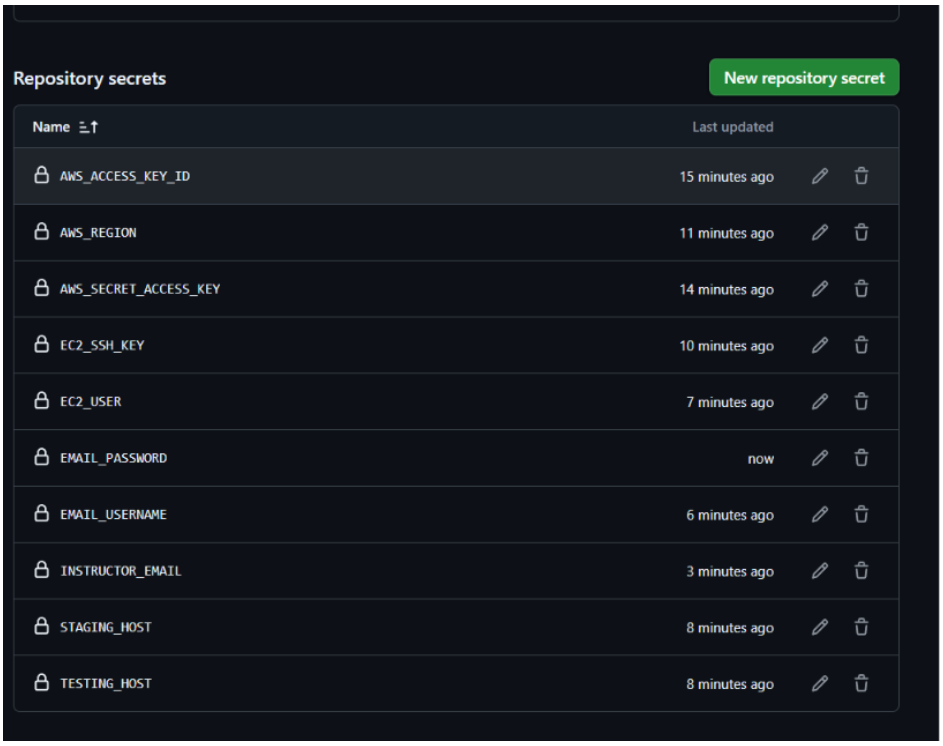

Figure 6: GitHub Actions Secrets page showing AWS and Email configuration.

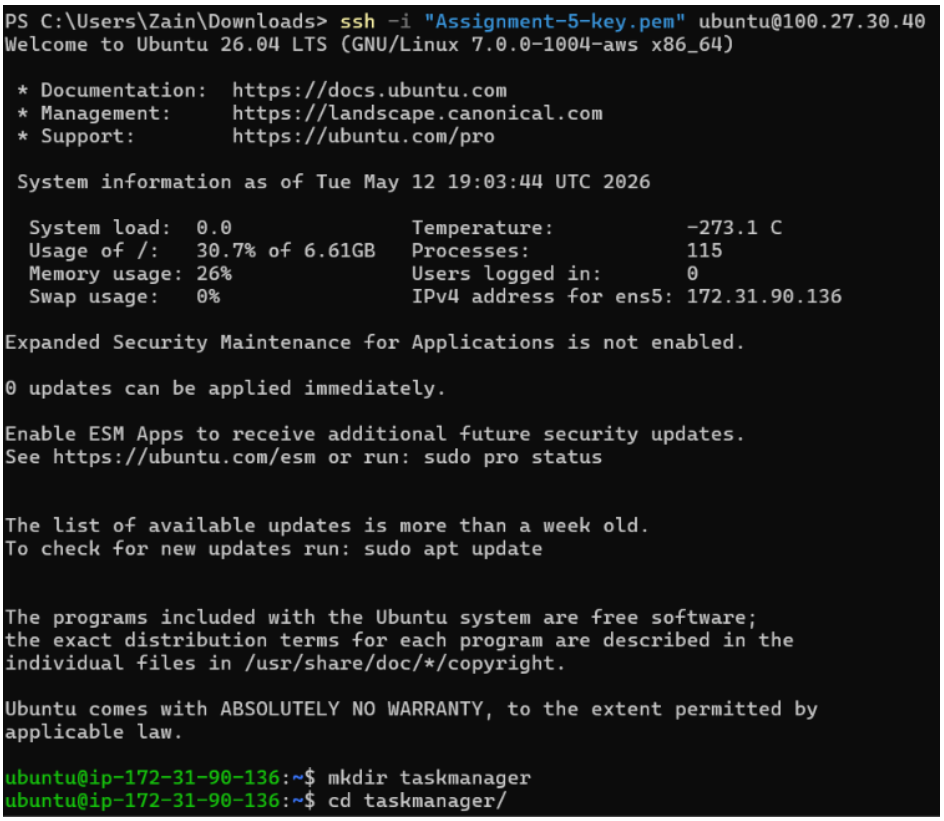

Figure 7: Testing EC2 terminal showing the production docker-compose.yml configured with ECR URIs.

Testing Environment Deployment Verification

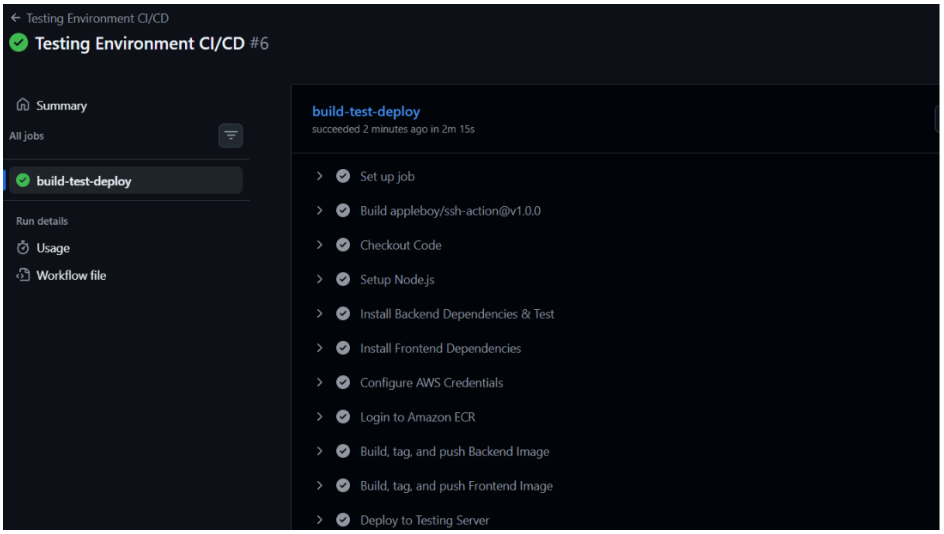

Figure 8: GitHub Actions tab showing the Testing workflow passing (green checkmarks).

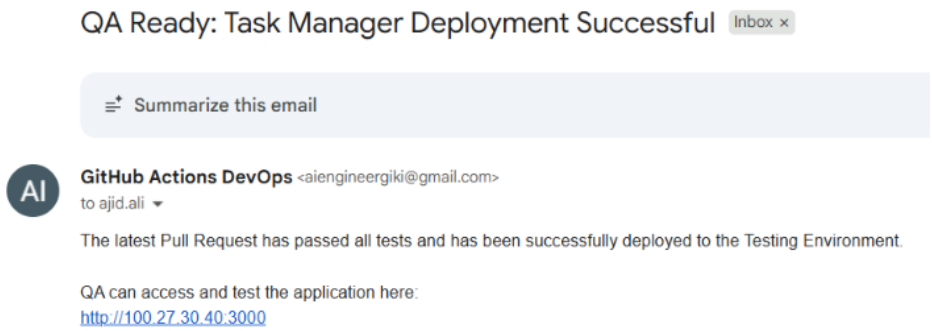

Figure 9: Automated QA success email showing the Testing server IP link.

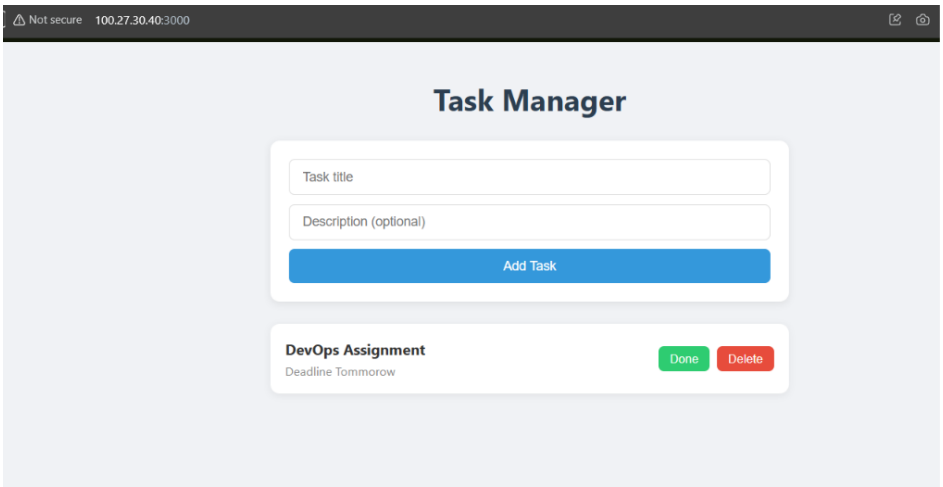

Figure 10: Browser showing the application running live on the Testing EC2 instance.

Staging Environment Post-Merge Deployment

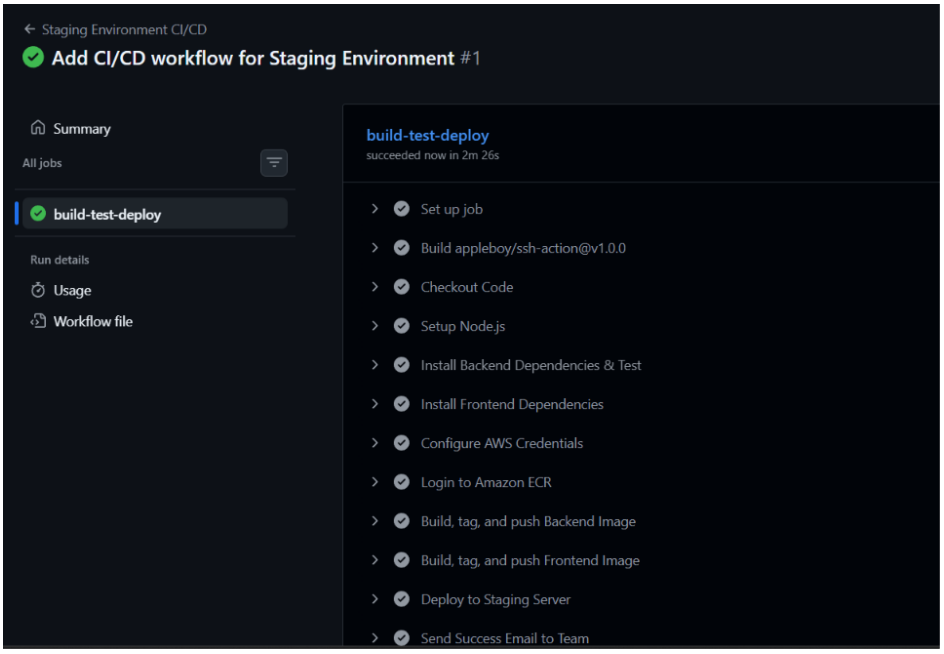

Figure 11: GitHub Actions tab showing the Staging workflow passing after merging the PR.

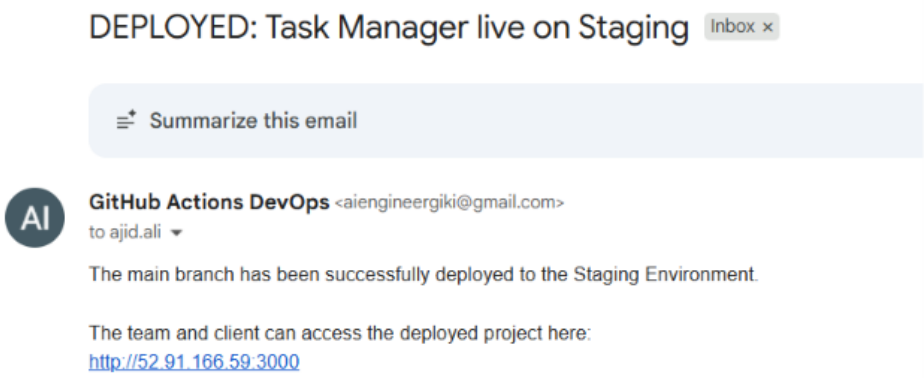

Figure 12: Automated Team success email showing the Staging server IP link.